

ICMP Covert Channel (40 points)

In this assignment, you will use a similar but even more covert technique than the one in the previous task. The network topology in this task is the same as before:



You need to transmit a message (string) from the victim to the attacker. This time, however, the firewall is even stricter — it does not allow any ICMP messages with an additional payload to pass through.

Environment

As before, the provided environment contains three hosts: the attacker container (192.168.124.10), the victim container (192.168.124.20), and the firewall, and you control both the attacker and the victim. Set up the environment as such:

```
docker compose build
docker compose up -d
```

Both containers have a mount configured; files in the `solution` directory are mirrored in the containers under `/solution`. You can enter the containers by executing the commands below in different terminal windows.

```
docker exec -ti assignment_3_covert_attacker bash
```

```
docker exec -ti assignment_3_covert_victim bash
```

After finishing the assignment, clean up the environment using the command below.

```
docker compose down
```

Goal

Create a Python3 script to establish communication between the two hosts and transfer a message. It should have two modes (passed as a command-line argument), `send` and `receive`, which run on the victim and on the attacker side respectively.

As the firewall does not allow ICMP packets with a payload to pass through, you need to find another way to sneak data through. For this, you can use some unused/unimportant parts of the IP or ICMP headers.

In this task, the use of encryption is optional. You may not disable or reconfigure the firewall, or use external libraries or tools different from those installed on the provided containers.

Sending. The first mode, **send**, should accept two additional arguments: the IP address of the receiver and the message to send. It should transmit the message and exit.

```
root@victim:/solution# python3 icmp_covert.py send 192.168.124.10 "Hello World!"
```

Receiving. The second mode, **receive**, should accept no additional arguments. It should listen for ICMP packets, decrypt the received message, print it, and exit.

```
root@attacker:/solution# python3 icmp_covert.py receive
Hello World!
```

Submission Instructions

After completing the assignment and verifying that it works correctly, submit your Python script on WebLab.